

● REC

HD

4K

25FPS



99%



08:07:20:26

Julia Ihrenberger
Sem. theor. Informatik

● REC

HD

4K

25FPS



99%

Die Anwendung von zellulären Automaten in der Bildverarbeitung

Vergleich und Analyse

08:07:20:26

Julia Ihrenberger
Sem. theor. Informatik

Analyse und **Vergleich**
von **vier Algorithmen**,
welche auf **zelluläre**
Automaten basieren

Einstiegs- und
Ansatzpunkt für weiter-
führende **Forschung** in
diesem Bereich

Salt and Pepper Noise (SPN)

- verursacht durch defekte Kamerasensoren, Übertragungsfehler bei Datentransfer, Bildfehler nach Aufbewahrung/Abruf
- Zufälliges Ersetzen von Pixelwerten durch Min (0) oder Max (255)
- führt zu Informationsverlust oder -verfälschung im Bild



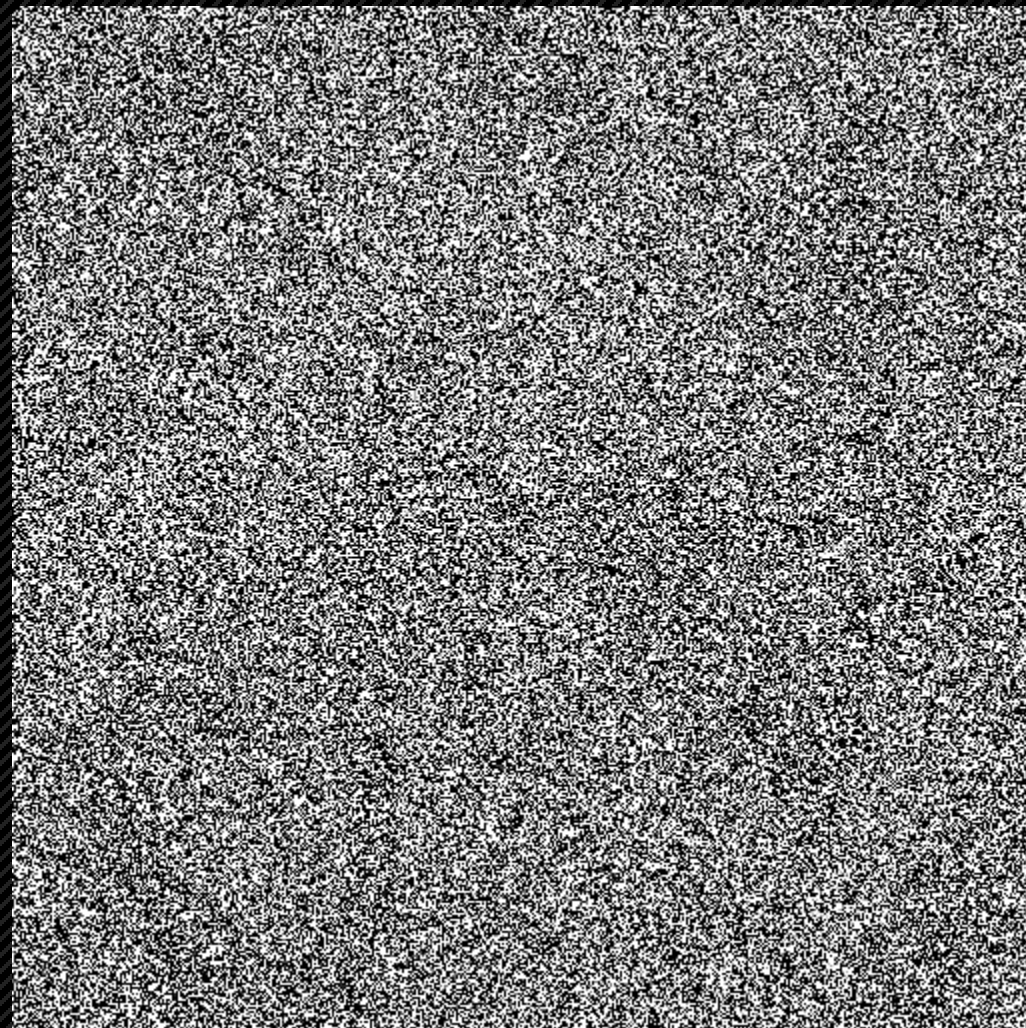
Rauschdichte: 10%



Rauschdichte: 50%



Rauschdichte: 90%



Rauschdichte: 20%

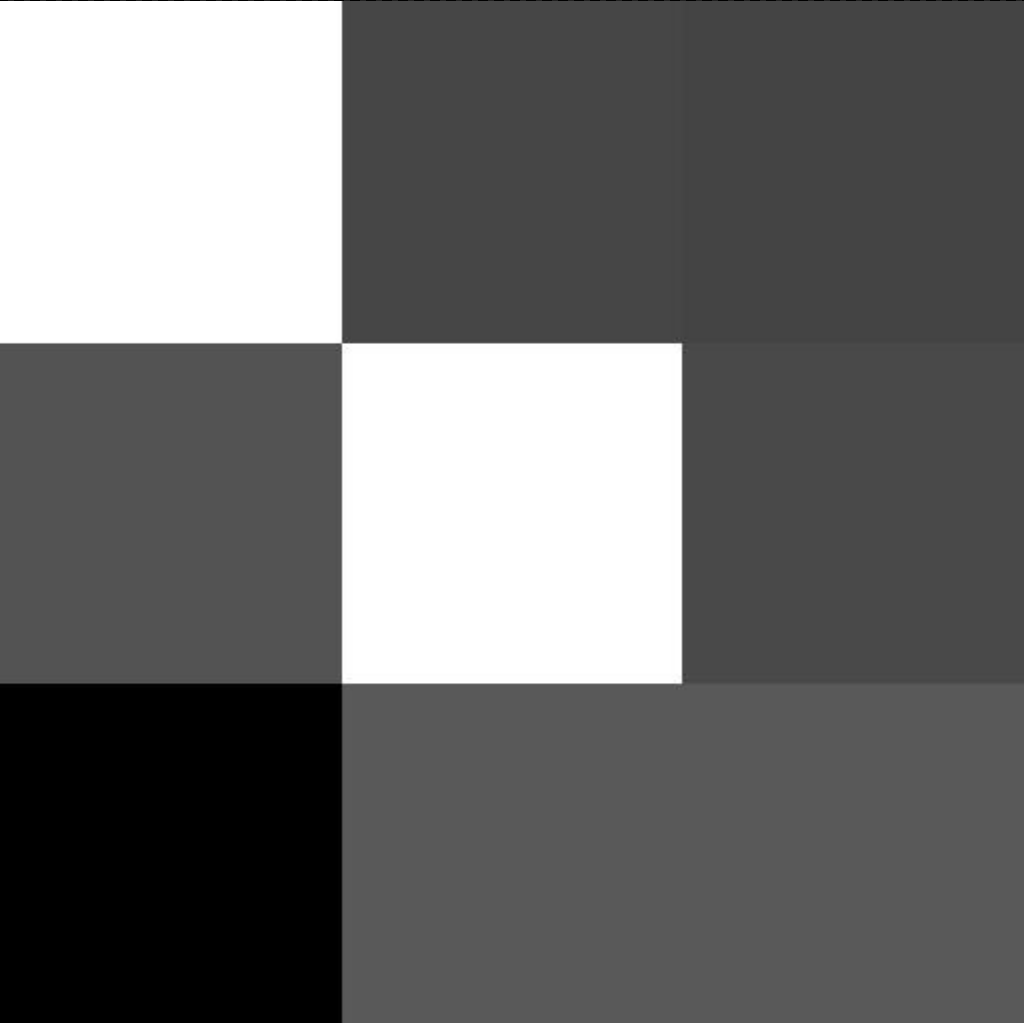
Rauschunterdrückung von SPN:

Median Filter

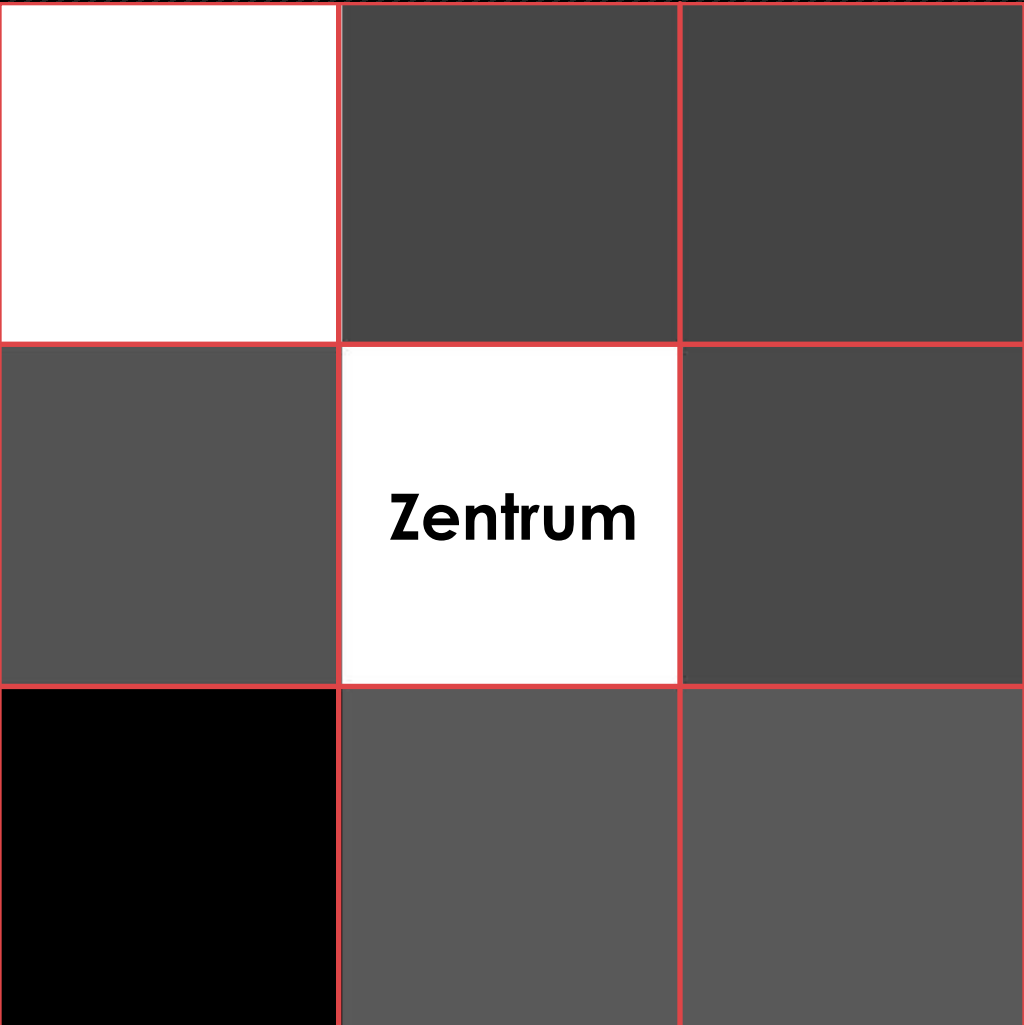
- jeder Pixel im Bild wird durch den Median seiner Nachbarschaft ersetzt
- dient oft als Vergleichsverfahren für SPN Entrauschen



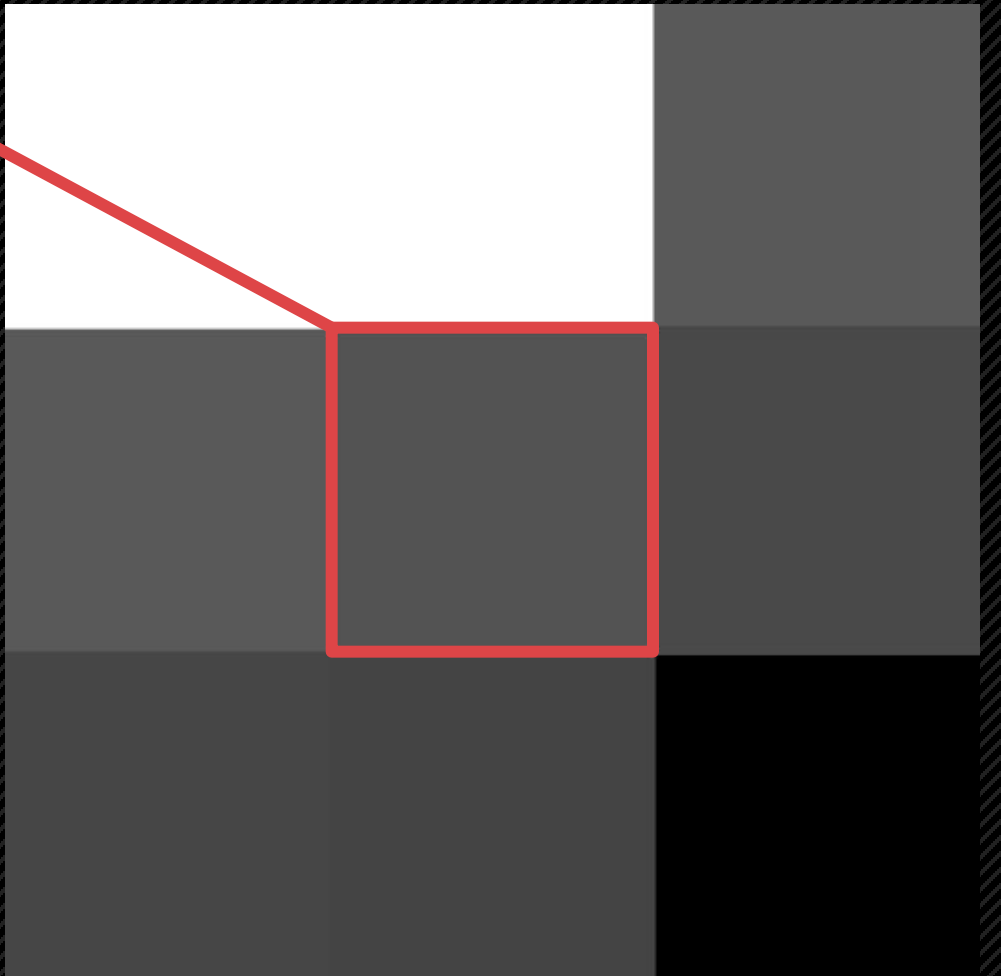
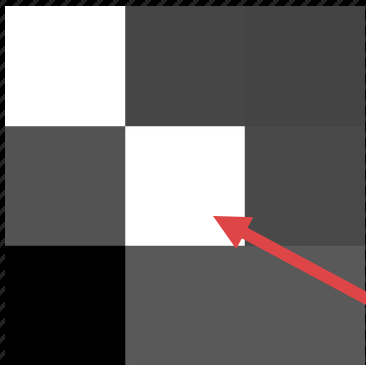
- Betrachtung von **3x3 Fenster** um Zentrum



- Betrachtung von **3x3 Fenster** um Zentrum



- sortiert Pixelwerte der Größe nach **absteigend**
- ersetzt **Zentrum** durch **Median**





10%



Verrauscht



restauriert über Digimizer Median



50%



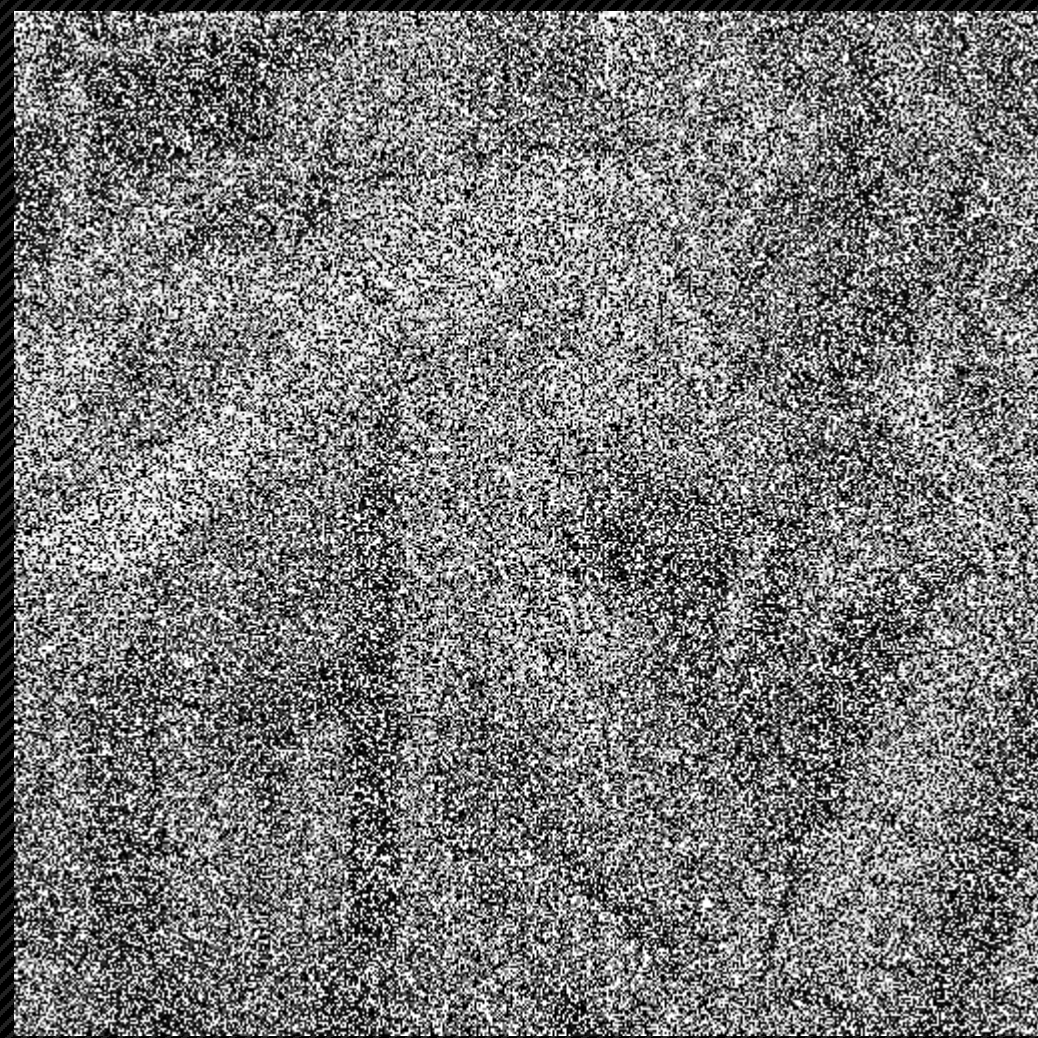
Verrauscht



restauriert über Digimizer Median



70%



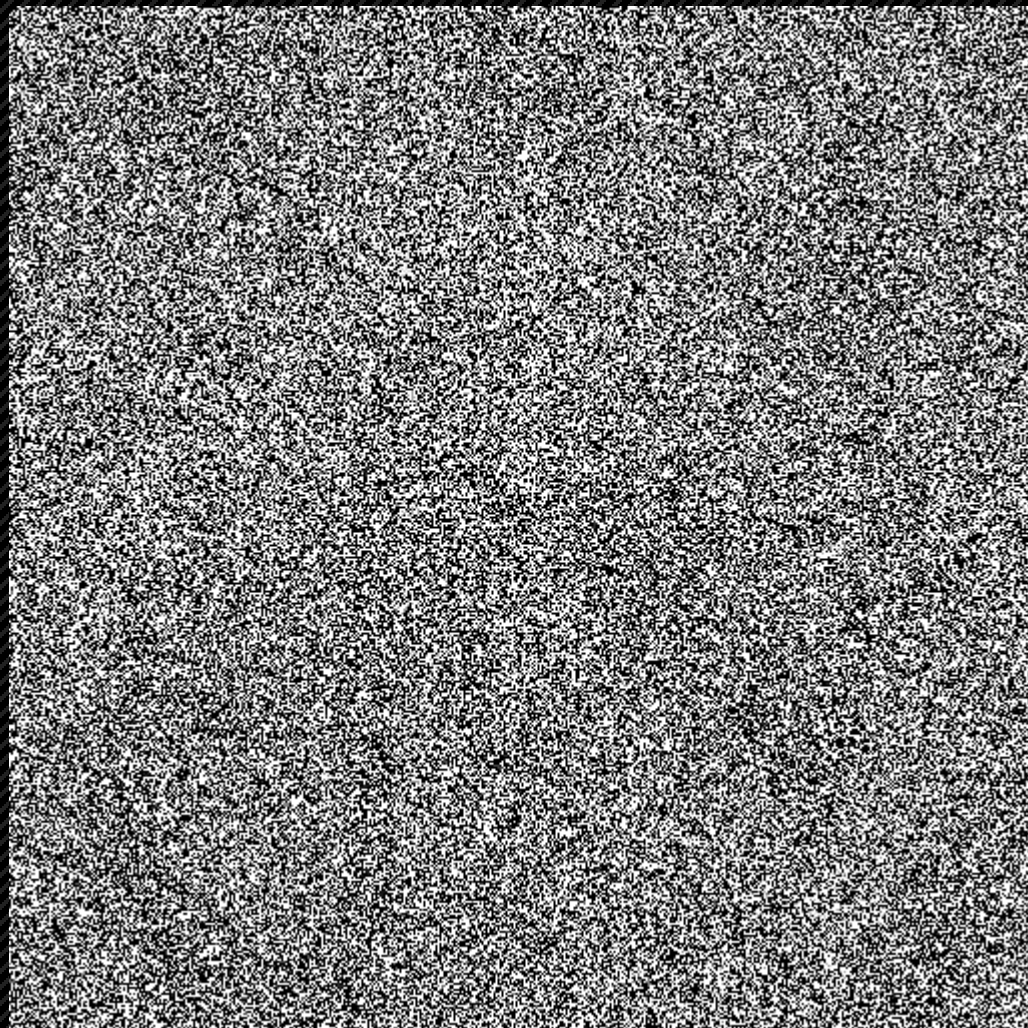
Verrauscht



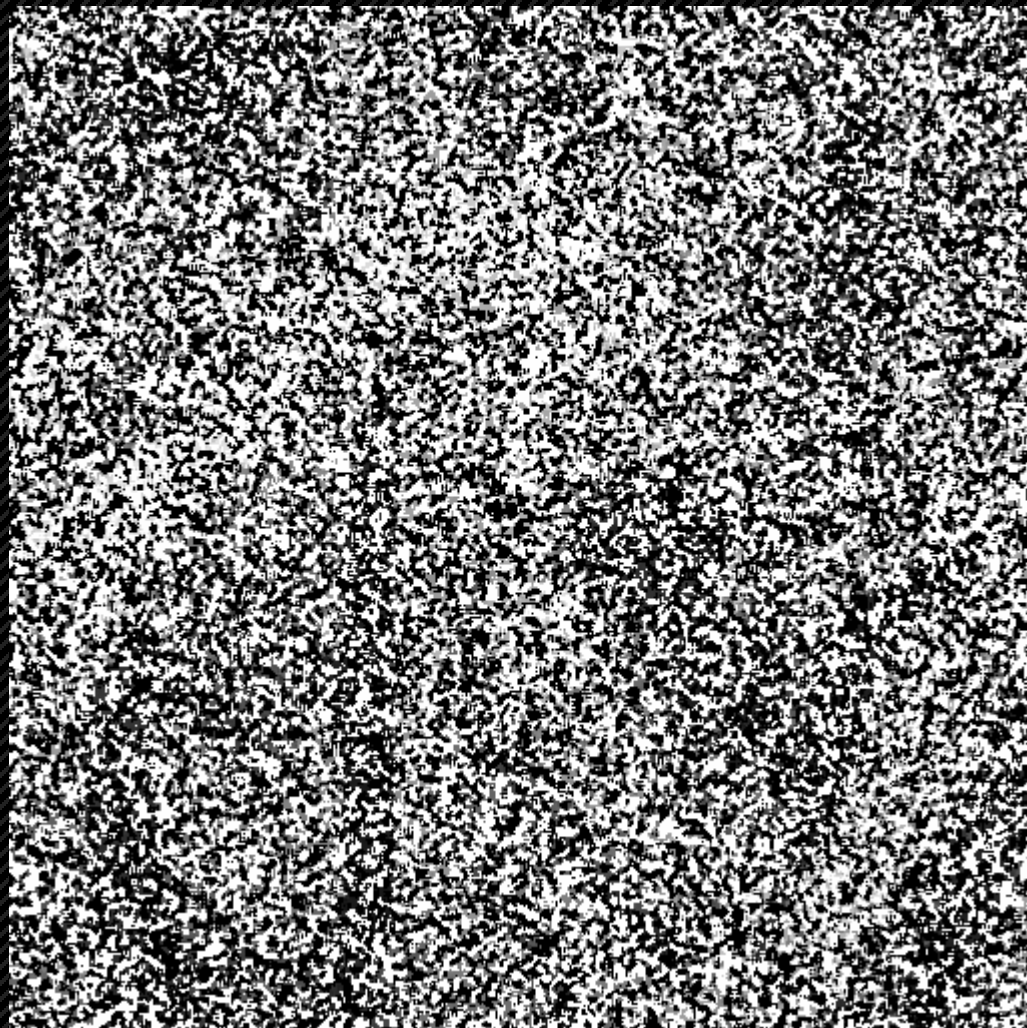
restauriert über Digimizer Median



90%



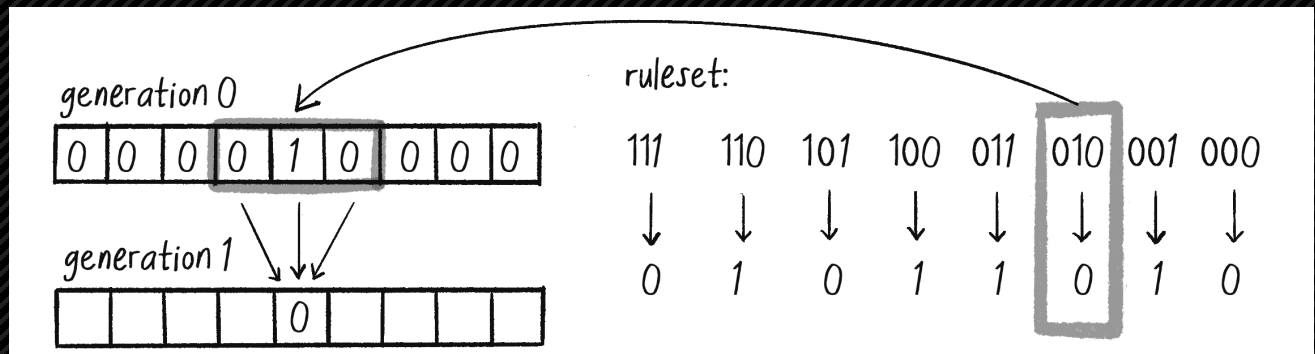
Verrauscht



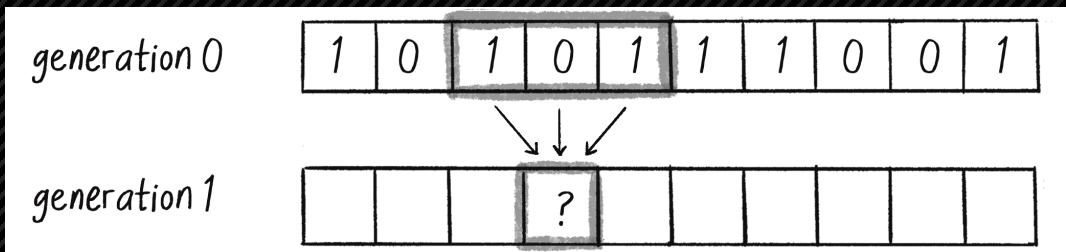
restauriert über Digimizer Median

Grundlegende Charakteristika von zellulären Automaten (CA)

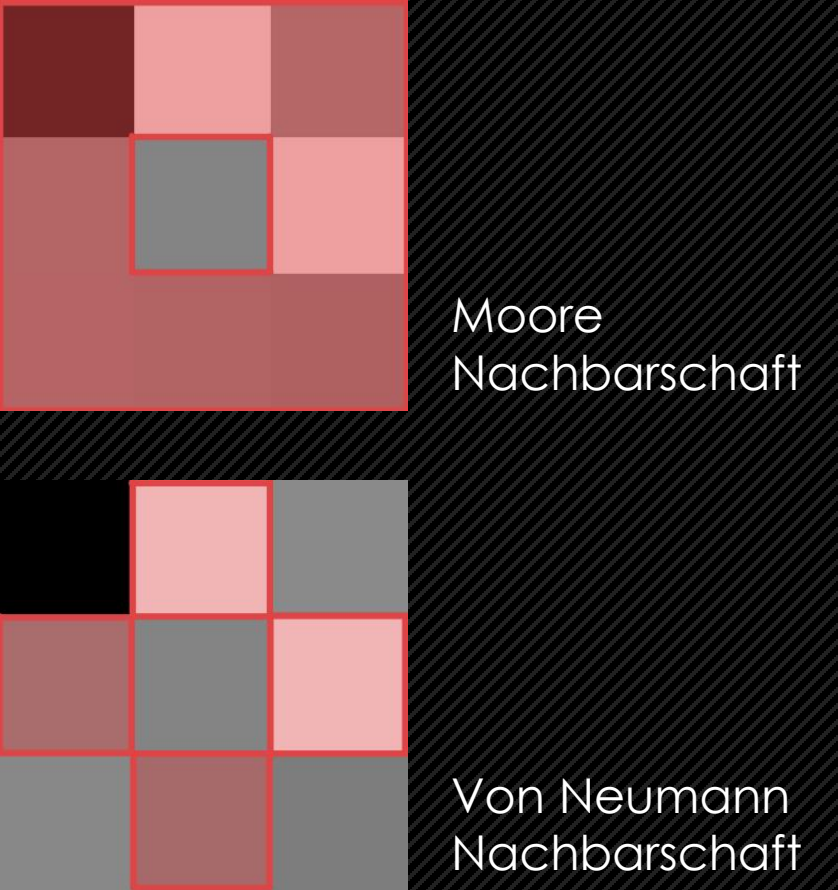
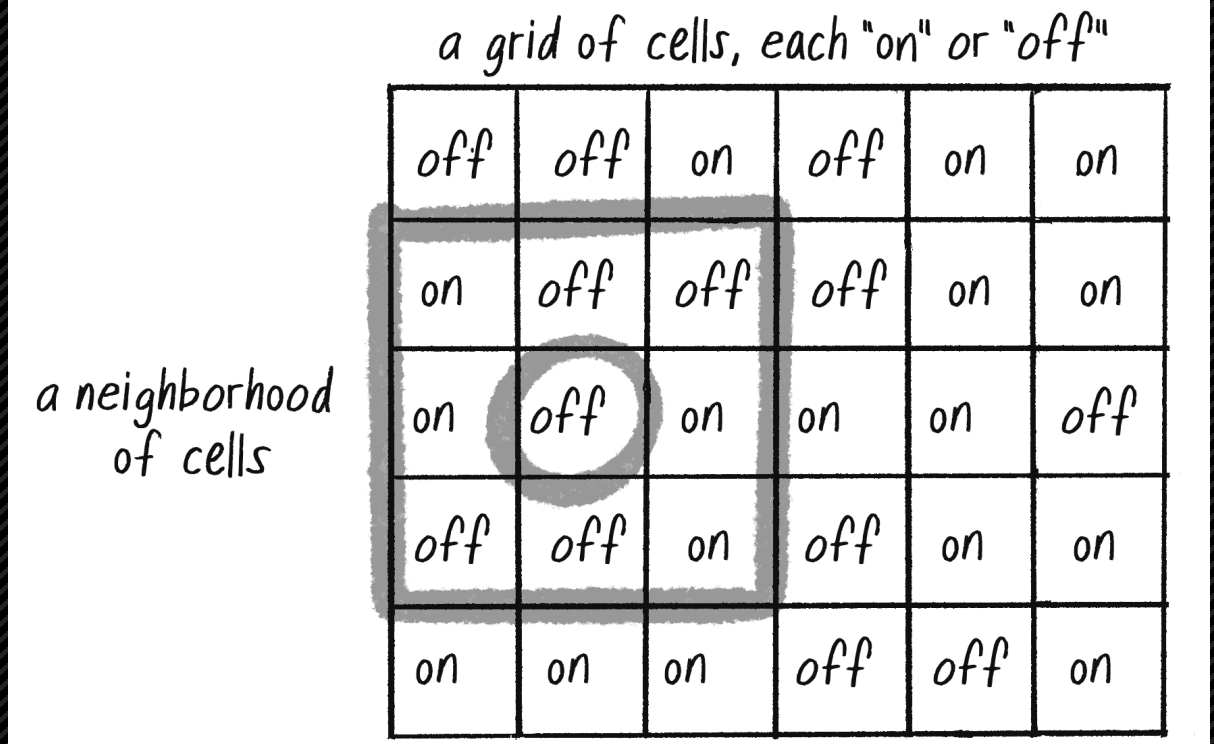
- Betrachtung von Zellen in einem Raum
- Jede Zelle in einem Zustand
- Jede Zelle besitzt eine Nachbarschaft
- Übergangsfunktion zur Änderung der Zelle bei Generationsschritt
- Ausgabe vom CA wird beeinflusst von Ausgangssituation und Übergangsfunktion



111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
0	1	0	1	1	0	1	0



gen 0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
gen 1	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
gen 2	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0
gen 3	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	0
gen 4	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0
gen 5	0	0	0	1	0	1	0	0	0	0	0	1	0	1	0	0	0
gen 6	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0
gen 7	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
gen 8	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1



Beispiel von Daniel Shiffman

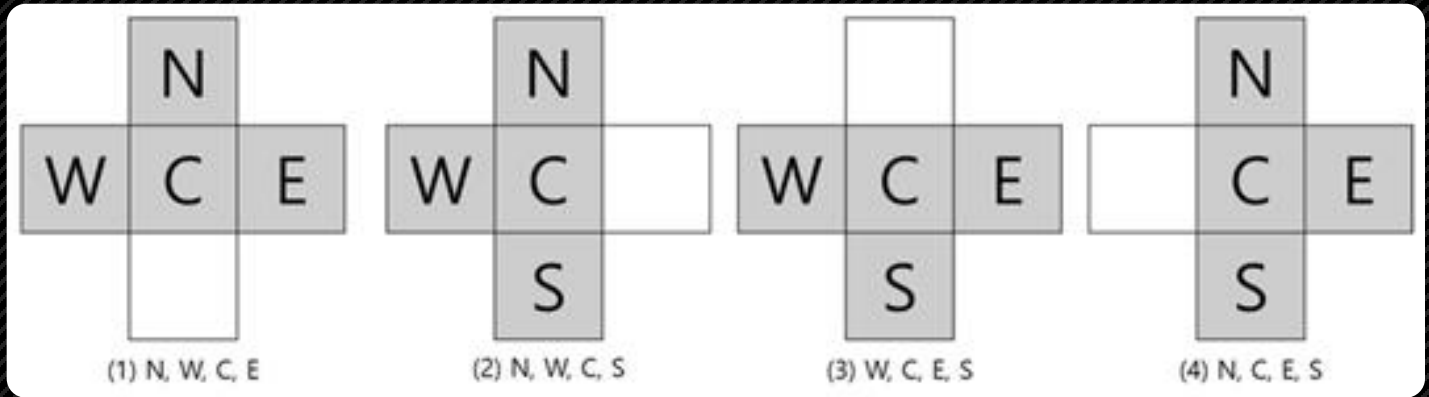
- Zweidimensionaler Gitterraum
- Zustände: on (1), off (0)
- Moore Nachbarschaft

Noise	Filter	Cell. Autom.	KCA	MERIG	SRKA	JQA	Ana- lyse	Fazit	
-------	--------	-----------------	-----	-------	------	-----	--------------	-------	--

CA in der Bildverarbeitung

- naheliegende Darstellung von Bildern als 2D CA
- Kapazität zu komplexen Verhalten, Kleine Änderungen = große Auswirkung
- begrenzte Effektivität des Median bei hohen Rauschdichten
- mehrere Werke mit CA-basierten Filtern übertrafen herkömmliche Filtermethoden

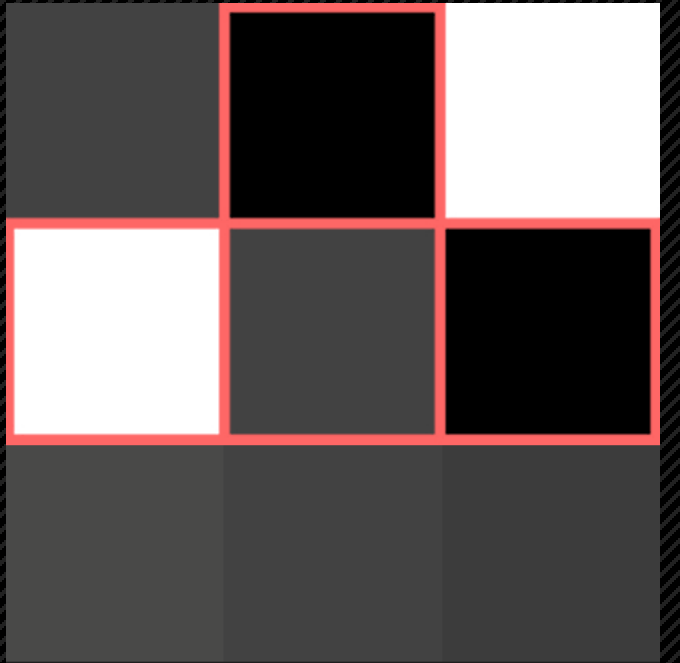
G. Kong, U. Choi: Reduzierte Betrachtung der Nachbarschaft



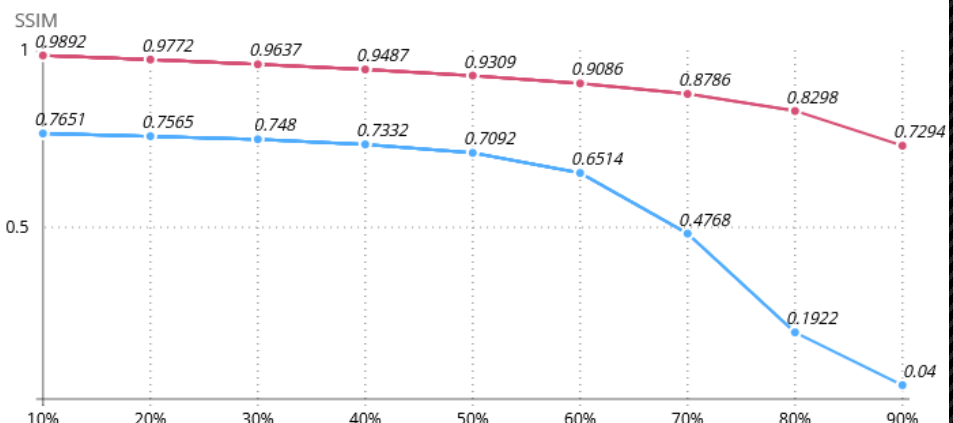
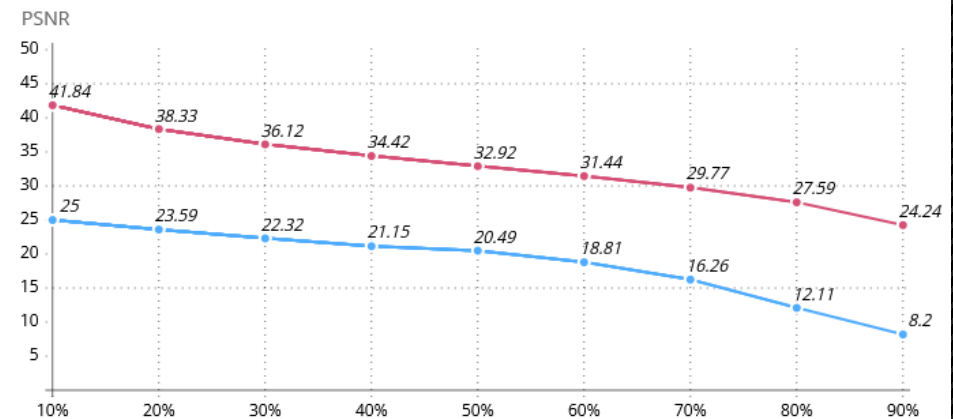
KCA Verfahren

```

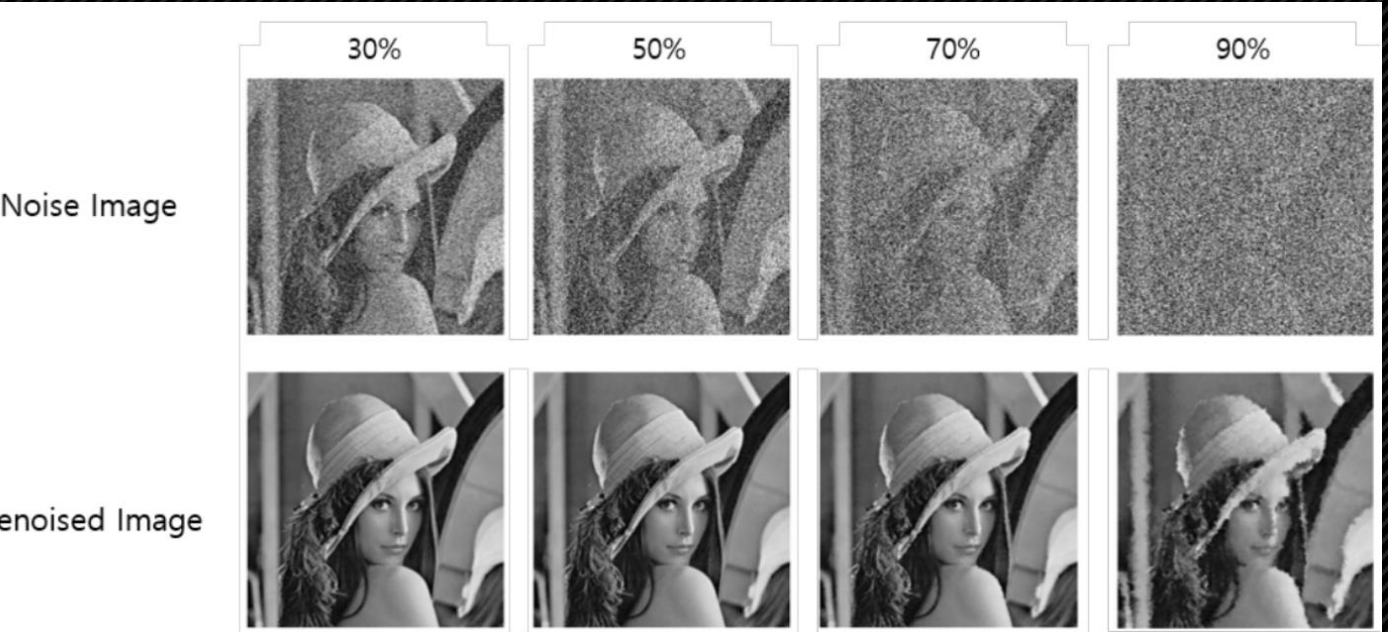
1: Input: Noisy image  $I$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x_i$  in  $I$  such that  $x_i = 0$  or  $255$ 
4: Repeat
5:   for each noisy pixel do
6:     for each neighbor group  $G_k$  in order ( $G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow G_4$ ) do
7:       Count the number of non-noisy pixels  $n$  in  $G_k$ 
8:       if  $2 \leq n$  then
9:         Replace  $x_i$  with the mean of non-noisy pixels in  $G_k$ 
10:        break
11:      end if
12:    end for
13:  end for
14: until no noisy pixels remain
  
```



- Identifikation über **Extremwerte**
- **Reduzierte Von Neumann** Nachbarschaft
- Zentrum wird ersetzt durch **Durchschnittswert** aller **sauberen** Pixel



Noise (%)	10	20	30	40	50	60	70	80	90
Iterations	2	4	4	6	6	9	15	28	69



- **signifikante Verbesserung** in Leistung
- **gezielte Eliminierung** von nur verrauschten Pixel
- **aber:** keine allgemeingültige Angabe zu benötigten Anwendungsdurchläufen
- gegebene Richtwerte gelten für **Lena Testbild**

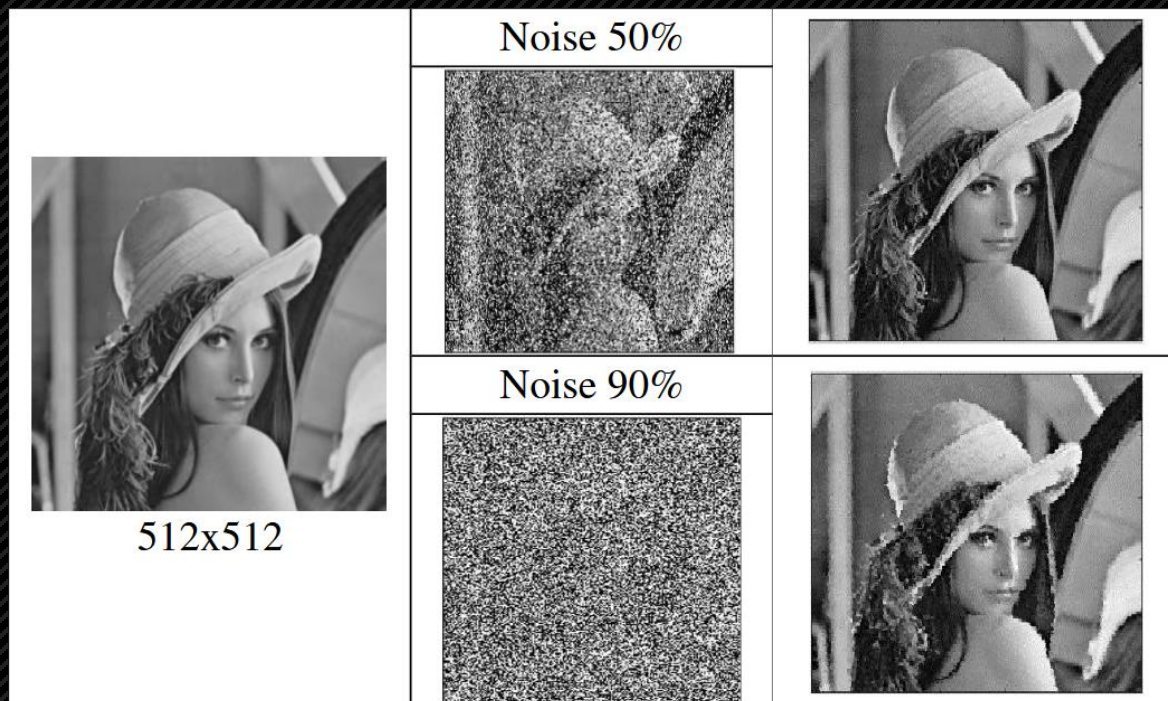
K. Angulo-Sogamoso, D. Gil-Sierra
and H. Espitia-Cuchango:
Adaptive Betrachtung der Nachbarschaft

MERIG Algorithmus (gemäß der Beschreibung in [18])

```

1: Input: Noisy image  $I$  and noise level  $n$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x$  in  $I$  such that  $x = 0$  or  $255$ 
4: Repeat
5:   for each noisy pixel  $x$  do
6:     count number of non-noisy pixels  $n$  in neighborhood  $N$ 
9:     if  $n == 0$  and  $d \neq 4$  then
10:      increase neighborhood size  $d$  to 4 and go to 6
12:    else
13:      replace  $x_i$  with the mean of non-noisy pixels in  $N$ 
14:    break
15:    end if
16:  end for
17: end for
18: until iteration count =  $\frac{n}{10} + 1$ 

```



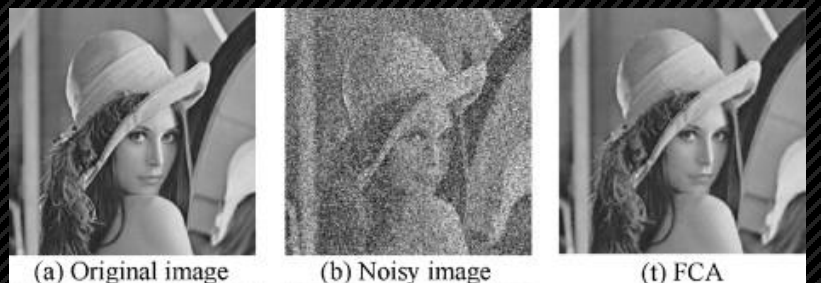
- gegensätzlicher Ansatz in der Betrachtung der Nachbarschaft (adaptiv)
 - merkliche Auswirkung auf Leistung zwischen KCA und MERIG
- Anzahl Durchläufe abhängig von Rauschlevel

S. Sadeghi, A. Rezvanian, E. Kamrani:
Zelluläre Automaten mit Fuzzy Logic

SRKA

```

1: Input: Noisy image  $I$ 
2: Output: Denoised image  $I'$ 
3: Identify  $x_{i,j}$  in  $I$  such that  $F(i,j) \geq \theta$ 
4: Repeat
5:   for each noisy pixel  $x_{i,j}$  do
6:     for each pixel  $y_i$  in Von Neumann neighborhood do
7:       fuzzify  $y_i = M_i(y)$ 
8:     end for
9:     replace  $x_{i,j}$  with average center of gravity  $Y$ 
10:  end for
11: until  $I$  is in a stable state
  
```



50% Rauschdichte

$$F(i,j) = \frac{N_m}{N_s} \quad (4)$$

$$N_m = \frac{1}{n} \sum_{i=1}^n X_i \quad (5)$$

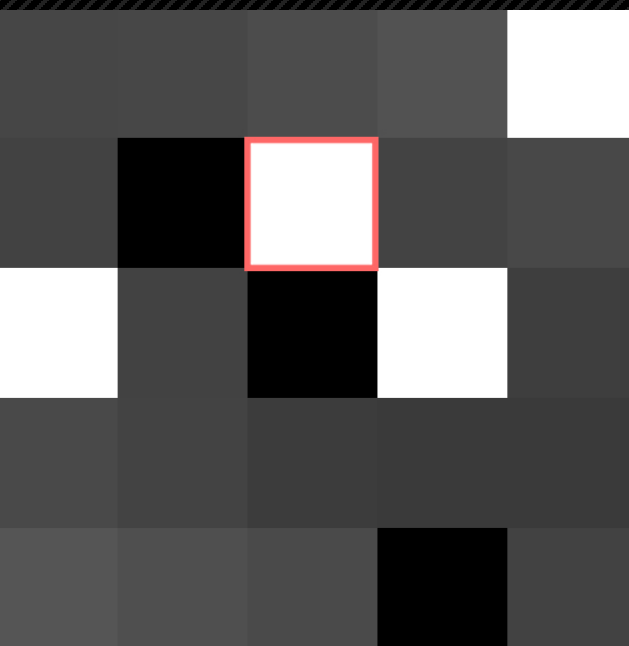
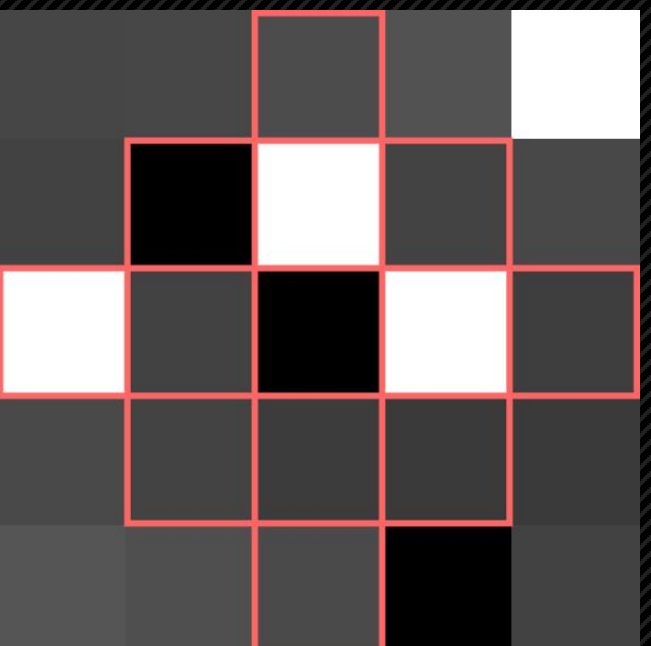
$$N_s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (D_i - \bar{D})^2} \quad (6)$$

$$D_i = |x_{center} - x_i|, \quad 1 \leq i \leq 8 \quad (7)$$

- Identifikation des Rauschens über Standardabweichung der Nachbarpixel in Relation zum Durchschnitt

- Restauration über Fuzzy Logic
 - Fuzzy Darstellung der Nachbarn anhand allg. Durchschnittswert (127.5)
 - Defuzzifizierung durch Berechnung des durchschnittlichen Schwerpunkts der Nachbarn
- bessere Leistung als Median, aber keine Angabe zu nötigen Iteration

ψ_{21}	ψ_{13}	ψ_9	ψ_{14}	ψ_{22}
ψ_{15}	ψ_5	ψ_1	ψ_6	ψ_{16}
ψ_{10}	ψ_2	$C(i,j)$	ψ_3	ψ_{11}
ψ_{17}	ψ_7	ψ_4	ψ_8	ψ_{18}
ψ_{23}	ψ_{19}	ψ_{12}	ψ_{20}	ψ_{24}



Z. Jeelani, F. Qadir:
Direktes Ersetzen
des Zentrums

```

JQA
1:  Input: Noisy image C
2:  Output: Denoised image C'
3:  for all pixel (i,j) in C in reverse order do
4:    if C(i,j) = 0 or 255 then
5:      for k ← 1 to 12 do
6:        if  $\psi_k(C(i, j)) \neq 0$  and 255 then
7:           $C(i',j') = \psi_k(C(i, j))$ 
8:          break
9:        end if
10:      end for
11:    end if
12:  end for
  
```

- Von Neumann Nachbarschaft mit zweifacher Distanz
- Suche nach sauberem Pixel in geordneter Reihenfolge
- Restauration des Zentrums durch Übernehmen des Werts



- Filter 2 repräsentativ für das Verfahren (höchste Effektivität)
- Überraschende Leistung auch bei hohen Rauschdichten
 - Demonstriert vorhandene Informationsdichte
- starke schlieren-ähnliche Artefakte bei besonders hohem Rauschlevel
- keine Angabe zu Iterationsanzahl

KCA Verfahren

```

1: Input: Noisy image  $I$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x_i$  in  $I$  such that  $x_i = 0$  or  $255$ 
4: Repeat
5:   for each noisy pixel do
6:     for each neighbor group  $G_k$  in order ( $G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow G_4$ ) do
7:       Count the number of non-noisy pixels  $n$  in  $G_k$ 
8:       if  $2 \leq n$  then
9:         Replace  $x_i$  with the mean of non-noisy pixels in  $G_k$ 
10:        break
11:      end if
12:    end for
13:  end for
14: until no noisy pixels remain
  
```

MERIG Algorithmus (gemäß der Beschreibung in [18])

```

1: Input: Noisy image  $I$  and noise level  $n$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x$  in  $I$  such that  $x = 0$  or  $255$ 
4: Repeat
5:   for each noisy pixel  $x$  do
6:     count number of non-noisy pixels  $n$  in neighborhood  $N$ 
9:     if  $n == 0$  and  $d \neq 4$  then
10:      increase neighborhood size  $d$  to 4 and go to 6
12:    else
13:      replace  $x_i$  with the mean of non-noisy pixels in  $N$ 
14:    break
15:  end if
16:  end for
17: end for
18: until iteration count  $= \frac{n}{10} + 1$ 
  
```

Noise	Filter	Cell. Autom.	KCA	MERIG	SRKA	JQA	Ana- lyse	Fazit	
-------	--------	-----------------	-----	-------	------	-----	--------------	-------	--

SRKA

```

1: Input: Noisy image  $I$ 
2: Output: Denoised image  $I'$ 
3: Identify  $x_{ij}$  in  $I$  such that  $F(i,j) \geq \theta$ 
4: Repeat
5:   for each noisy pixel  $x_{ij}$  do
6:     for each pixel  $y_i$  in Von Neumann neighborhood do
7:       fuzzify  $y_i = M_i(y)$ 
8:     end for
9:     replace  $x_{ij}$  with average center of gravity  $Y$ 
10:  end for
11: until  $I$  is in a stable state

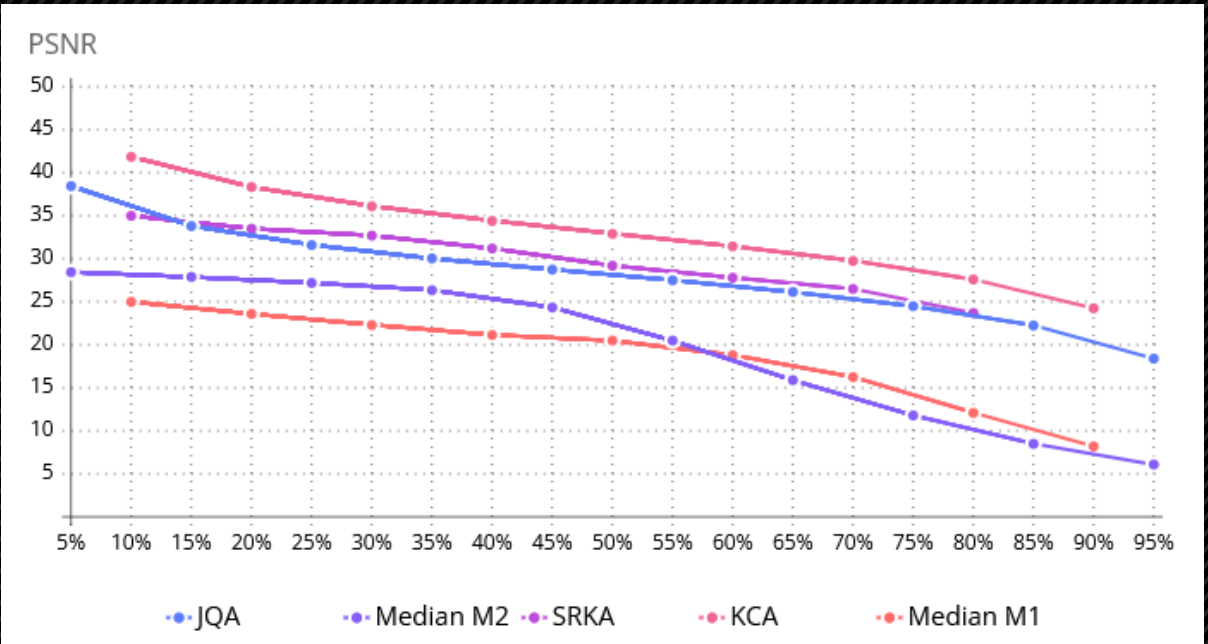
```

JQA

```

1: Input: Noisy image  $C$ 
2: Output: Denoised image  $C'$ 
3: for all pixel  $(i,j)$  in  $C$  in reverse order do
4:   if  $C(i,j) = 0$  or  $255$  then
5:     for  $k \leftarrow 1$  to  $12$  do
6:       if  $\psi_k(C(i,j)) \neq 0$  and  $255$  then
7:          $C(i',j') = \psi_k(C(i,j))$ 
8:         break
9:       end if
10:    end for
11:   end if
12: end for

```



KCA Verfahren

- 1: **Input:** Noisy image I
- 2: **Output:** Denoised image I'
- 3: Identify noisy pixels x_i in I such that $x_i = 0$ or 255
- 4: **Repeat**
- 5: **for** each noisy pixel **do**
- 6: **for** each neighbor group G_k in order ($G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow G_4$) **do**
- 7: Count the number of non-noisy pixels n in G_k
- 8: **if** $2 \leq n$ **then**
- 9: Replace x_i with the mean of non-noisy pixels in G_k
- 10: **break**
- 11: **end if**
- 12: **end for**
- 13: **end for**
- 14: **until** no noisy pixels remain

MERIG Algorithmus (gemäß der Beschreibung in [18])

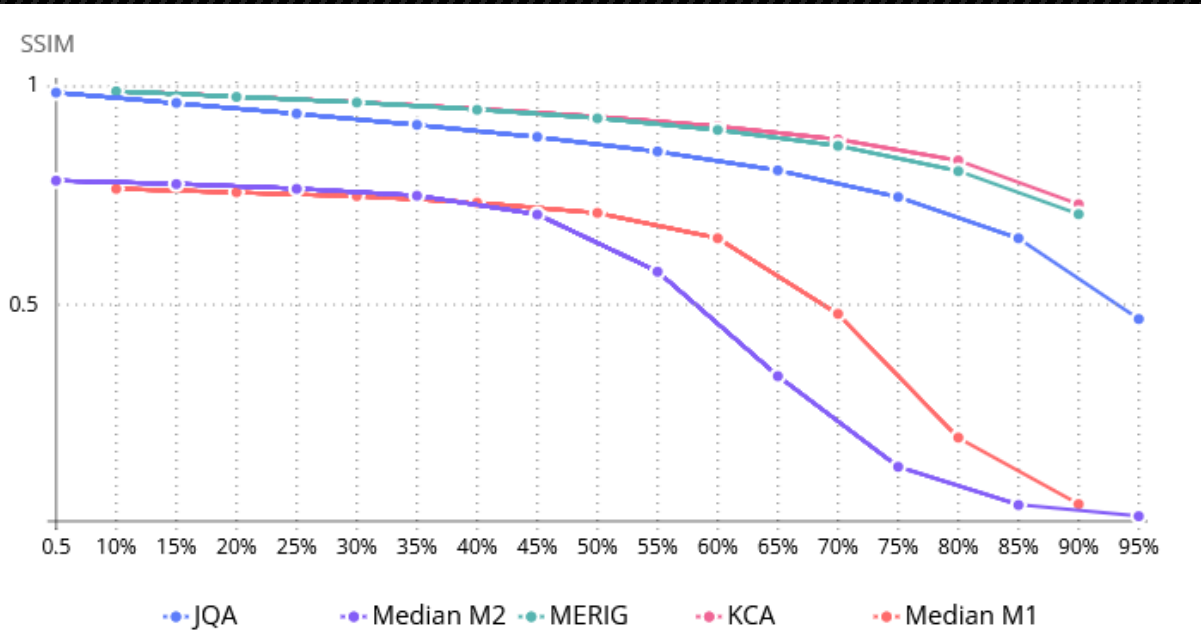
- 1: **Input:** Noisy image I and noise level n
- 2: **Output:** Denoised image I'
- 3: Identify noisy pixels x in I such that $x = 0$ or 255
- 4: **Repeat**
- 5: **for** each noisy pixel x **do**
- 6: count number of non-noisy pixels n in neighborhood N
- 9: **if** $n == 0$ and $d \neq 4$ **then**
- 10: increase neighborhood size d to 4 and go to 6
- 12: **else**
- 13: replace x_i with the mean of non-noisy pixels in N
- 14: **break**
- 15: **end if**
- 16: **end for**
- 17: **end for**
- 18: **until** iteration count = $\frac{n}{10} + 1$

SRKA

- 1: **Input:** Noisy image I
- 2: **Output:** Denoised image I'
- 3: Identify $x_{i,j}$ in I such that $F(i,j) \geq \theta$
- 4: **Repeat**
- 5: **for** each noisy pixel $x_{i,j}$ **do**
- 6: **for** each pixel y_i in Von Neumann neighborhood **do**
- 7: fuzzify $y_i = M_i(y)$
- 8: **end for**
- 9: replace $x_{i,j}$ with average center of gravity Y
- 10: **end for**
- 11: **until** I is in a stable state

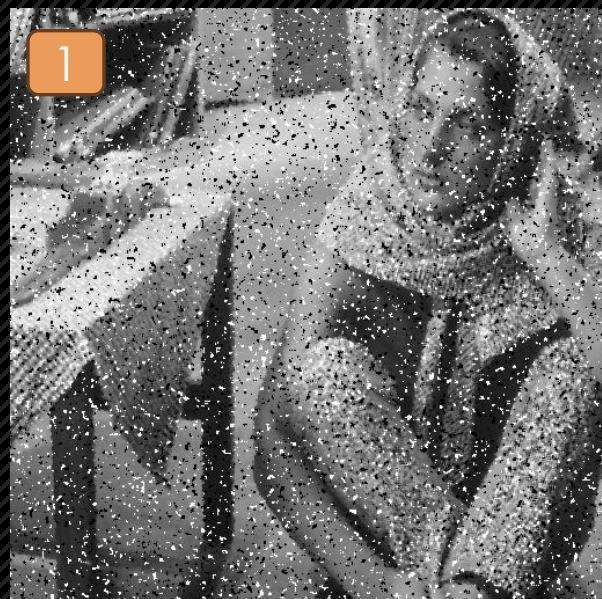
JQA

- 1: **Input:** Noisy image C
- 2: **Output:** Denoised image C'
- 3: **for** all pixel (i,j) in C in reverse order **do**
- 4: **if** $C(i,j) = 0$ or 255 **then**
- 5: **for** $k \leftarrow 1$ to 12 **do**
- 6: **if** $\psi_k(C(i,j)) \neq 0$ and 255 **then**
- 7: $C(i',j') = \psi_k(C(i,j))$
- 8: **break**
- 9: **end if**
- 10: **end for**
- 11: **end if**
- 12: **end for**





- Anwendung von CA in der Bildverarbeitung weist **hohe Effektivität** auf
- Größte Aufwertungen durch **Rausch-Identifikation** und **iterativer Anwendung**
 - vgl. Medianfilter mit 1, 2, 3 und 12 Durchgängen, aber **Degradierung der Details**
- Wenig farbliche Bildverarbeitung** sowie **andere Arten von Rauschen**



**Vielen Dank für Eure
Aufmerksamkeit!**

22112186

OLAROID

Noch Fragen?

22112186

OLAROID

- **Babara Testbild:** <https://ccia.ugr.es/cvg/CG/base.htm>
- **Camera Frame:** https://www.magnific.com/free-vector/camera-viewfinder-frame_5093632.htm#fromView=keyword&page=1&position=0&uuid=ffbfb5f1-f8ce-4797-8bcb-c48aa825d9e0&query=Camera+frame
- **Polaroid Frame:** <https://de.vecteezy.com/png/43848068-retro-sammelalbum-polaroid-rahmen-klassisch-sofortig-foto-rand-zum-kreativ-collagen-und-erinnerung-behalten>
- **Camera Film:** https://www.magnific.com/free-vector/realistic-vector-icon-film-tape-strip-with-white-square-isolated-white-cinema-concept_31096470.htm#fromView=keyword&page=1&position=0&uuid=bca25bf4-9748-4eb2-b3af-e9b4a95791b5&query=Camera+film+png
- **Noise:** Z. Ansari, "Understanding Image Noise: Exploring Different Kinds of Noises in Image Processing," 4 February 2025. [Online]. Available: https://medium.com/@zar_373/understanding-image-noise-exploring-different-kinds-of-noises-in-image-processing-dcd26c6c9e5a.
- **Median Filter:** University of Edinburgh, "Spatial Filters - Median Filter," 2020. [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/HIPR2/median.htm>. [Accessed 24 June 2026].
- R. C. Gonzales and R. E. Woods, "Restoration in the Presence of Noise Only - Spatial Filtering," in Digital Image Processing, New Jersey, Prentice Hall, 2008, pp. 322-335.
- **KCA:** G. Kong and U. Choi, "A cellular automata-based method for salt-and-pepper noise removal in images," Multimedia Tools and Applications, vol. 85, no. 3, 2026.
- **MERIG:** K. Angulo-Sogamoso, D. Gil-Sierra and H. Espitia-Cuchango, "A NOVEL ALGORITHM BASED ON CELLULAR AUTOMATA TO ELIMINATE NOISE IN DIGITAL IMAGES," ARPN Journal of Engineering and Applied Sciences, vol. 14, no. 18, pp. 3289-3300, 2019.
- **SRKA:** S. Sadeghi, A. Rezvanian and E. Kamrani, "An efficient method for impulse noise reduction from images using fuzzy cellular automata," AEU - International Journal of Electronics and Communications, vol. 66, no. 9, pp. 772-779, 2012.
- **JQA:** Z. Jeelani and F. Qadir, "Cellular automata-based approach for salt-and-pepper noise filtration," Journal of King Saud University - Computer and Information Sciences, vol. 34, no. 2, pp. 365-374, December 2018.
- Jegliche Graphen und Testwerte aus oder basierend auf angegebenen Quellen
- Babara Testbild wurde mit <https://cocoshot.net/image-tools/add-noise/> verrauscht und mit **Digimizer** (MedCalc Software Ltd) Software restauriert